R1.02 — Développement d'interfaces web : Architecture Client-Serveur et Protocole HTTP

Thème principal : Fonctionnement fondamental du Web, architecture client-serveur, protocole HTTP, méthodes et codes de statut

Synthèse exécutive : Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur et le protocole HTTP. L'étudiant y découvre comment un navigateur formule des requêtes standardisées pour obtenir des ressources auprès d'un serveur web, comment ce dernier y répond de manière asynchrone et sans état (stateless), et comment décoder les codes de statut HTTP pour diagnostiquer et déboguer efficacement des applications web.

L'Architecture Client-Serveur et le Principe de Communication

L'architecture client-serveur est le modèle de communication fondamental du Web. Contrairement au pair-à-pair, les rôles sont figés. Le navigateur web de l'utilisateur (Firefox, Chrome, Safari) agit comme client en envoyant des requêtes HTTP à travers le réseau. Le serveur web (Apache, Nginx, IIS) écoute passivement les ports réseau. Lorsqu'il reçoit une requête, il la traite, va chercher les ressources nécessaires et renvoie une réponse HTTP contenant le fichier demandé (HTML, CSS, image, JSON) ou un message d'erreur.

- Modèle Client-Serveur : Distinction stricte entre le logiciel demandeur (client) et le processus fournisseur (serveur)
- Client (Navigateur) : Logiciel actif qui prend l'initiative d'envoyer une requête
- Serveur Web : Entité passive à l'écoute sur un port spécifique (ex: 80 pour HTTP, 443 pour HTTPS)
- Communication Asynchrone : Échange structuré où le client initie et le serveur répond pour clore la transaction

Cas concret enseignant : *Le professeur explique que lorsqu'un étudiant se connecte sur le site de l'IUT de Bayonne, son navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque image, feuille de style CSS et script JavaScript associé afin de reconstituer la page complète.*

Le Protocole HTTP et sa Nature Stateless (Sans État)

HTTP est un protocole sans état (stateless). Le serveur ne conserve aucune mémoire des requêtes précédentes d'un même client. Si cela allège la charge des serveurs et simplifie l'architecture, cela pose problème pour des applications interactives nécessitant un suivi (panier d'achat, authentification). Pour pallier cette limite, on utilise des cookies et des sessions.

- Protocole Stateless (Sans état) : Chaque échange requête-réponse est indépendant et autonome
- Absence de mémoire : Le serveur oublie instantanément l'interaction précédente avec le client
- Mécanismes de contournement : Utilisation des cookies (côté client) et des sessions (côté serveur) pour simuler une persistance

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un parfait inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase.*

Structure des Messages HTTP : Requêtes et Réponses

Une requête HTTP est un message textuel structuré comportant une ligne de requête, des en-têtes, une ligne vide et un corps optionnel. La réponse du serveur adopte une structure similaire mais commence par une ligne de statut. Le protocole HTTP est purement textuel, ce qui le rend lisible en clair en l'absence de chiffrement (HTTPS).

- Ligne de requête : Contient la méthode (verbe), l'URI et la version du protocole

• En-têtes (Headers) : Métadonnées contextuelles (Host, User-Agent)
• Ligne vide : Séparation obligatoire entre les en-têtes et le corps
• Corps (Body) : Données optionnelles transmises (principalement avec POST)
• Ligne de statut : Commence la réponse du serveur avec un code à trois chiffres et un message textuel

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel : on peut littéralement lire les requêtes et réponses en clair si elles ne sont pas chiffrées par HTTPS. C'est pourquoi l'utilisation de HTTP non sécurisé sur un réseau public expose toutes les données à n'importe quel attaquant équipé d'un sniffer de paquets.*

Les Méthodes HTTP Fondamentales : GET vs POST

La méthode GET est conçue pour récupérer des informations sans modifier l'état du serveur ; ses paramètres sont visibles dans l'URL. La méthode POST est conçue pour envoyer des données modifiant l'état du système (formulaires, insertions), transportées discrètement dans le corps (body) de la requête.

• Méthode GET : Consultation de ressource, sûre et idempotente, paramètres visibles dans l'URL
• Méthode POST : Soumission de données modifiant l'état du serveur, données encapsulées dans le corps de la requête

Cas concret enseignant : *Le professeur explique que l'utilisation de GET pour des données sensibles comme un mot de passe est une faille de sécurité majeure visible directement dans l'historique du navigateur ou sur l'écran de n'importe qui passant derrière l'utilisateur (shoulder surfing).*

La Classification des Codes de Statut HTTP

Les codes de statut HTTP permettent de catégoriser immédiatement le résultat d'une requête. Il est fondamental de distinguer les erreurs 4xx (faute du client : URL erronée, droits insuffisants) des erreurs 5xx (faute du serveur : bug de script, crash de base de données).

• Codes 2xx (Succès) : Requête reçue et traitée avec succès (ex: 200 OK)
• Codes 3xx (Redirection) : Actions supplémentaires requises (ex: 301 Moved Permanently)
• Codes 4xx (Erreur Client) : Anomalie provenant de la requête du client (ex: 404 Not Found, 403 Forbidden)
• Codes 5xx (Erreur Serveur) : Problème interne au serveur malgré une requête valide (ex: 500 Internal Server Error)

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs pensaient que le site était piraté alors que c'était simplement un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec GET, les identifiants et mots de passe apparaîtraient en clair dans l'URL (ex: ?user=tutu&pass;=1234), ce qui les expose dans l'historique et les logs. Avec POST, les données voyagent cachées dans le corps (body) de la requête HTTP, ce qui protège la vue directe, même s'il faut impérativement coupler cela avec HTTPS pour interdire l'interception réseau.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyse de la ligne de requête : La première ligne 'GET /images/logo.png HTTP/1.1' indique le verbe HTTP 'GET' (récupération de ressource), l'URI '/images/logo.png' et la version du protocole HTTP/1.1. 2. Identification de l'hôte :

L'en-tête 'Host: iutbayonne.univ-pau.fr' indique le domaine cible, indispensable pour le virtual hosting. 3. Analyse de l'agent utilisateur : 'User-Agent: Mozilla/5.0' identifie le type de navigateur client. 4. Scénario d'absence de ressource : Si le fichier 'logo.png' n'existe pas, le serveur ne peut satisfaire la demande du client et doit renvoyer un code d'erreur client de la famille 4xx, spécifiquement le code '404 Not Found'.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Code de retour si absent : 404 Not Found (Erreur Client)

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : C'est un protocole où chaque échange requête-réponse est indépendant et où le serveur ne conserve aucune mémoire des interactions passées avec le client.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les codes 4xx indiquent une erreur provenant du client (URL incorrecte, mauvaise syntaxe), tandis que les codes 5xx indiquent une défaillance interne du serveur alors que la requête était valide.

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que les paramètres de GET sont affichés en clair dans l'URL, ce qui les rend visibles dans l'historique du navigateur, les logs du serveur et par toute personne regardant l'écran par-dessus l'épaule de l'utilisateur.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765